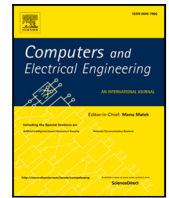




Contents lists available at ScienceDirect

## Computers and Electrical Engineering

journal homepage: [www.elsevier.com/locate/compeleceng](http://www.elsevier.com/locate/compeleceng)Locally enhanced denoising self-attention networks and decoupled position encoding for sequential recommendation<sup>☆</sup>Xingyao Yang<sup>ID</sup>\*, Xinsheng Dong, Jiong Yu, Shuangquan Li, Xinyu Xiong, Hongtao Shen

Xinjiang University, PR China

## ARTICLE INFO

## Keywords:

Sequential recommendation  
Attention mechanism  
Filtering algorithm  
Convolutional neural networks (CNNs)  
Multi-scale local preferences  
Local enhanced convolution

## ABSTRACT

Most of the existing Transformer-based models have been shown to have great advantages in sequential recommendation by modeling temporal dynamics through the self-attention mechanism. Nevertheless, the original self-attention mechanism requires the equal weighting and computation of all interactions between each item and every other item. This method presents limitations in effectively capturing shifts in users' local interests. In addition, this approach ignores the noise in user data, while absolute position encoding leads to inaccurate sequential relations between items. An innovative Locally Enhanced Denoising Self-Attention Network and Decoupled Position Encoding for Sequential Recommendation, named LEDADP, is presented to resolve these issues. Specifically, we use the noise filtering module **to convert the original data into the frequency domain to reduce the noise and achieve the purpose of filtering the noise.** We **integrate convolution into self-attention for local interest transfer, and we provide a multi-scale local enhanced convolution module that models local dependencies taking into account various local preferences at several scales, collecting more detailed local semantic information.** Furthermore, **in order to more precisely depict the sequential link between items, we additionally employ decoupled position encoding.** Extensive experiments conducted on three real-world datasets : Beauty, Toys, and ML-1M. The experimental results show that compared with the suboptimal model, the proposed model has respectively improved by 2.87%, 7.83% and 4.1% on Recall@5, and by 2.03%, 4.08% and 6.8% on NDCG@5, which proves the validity of the model.

## 1. Introduction

Large streaming services like Netflix and e-commerce sites like Amazon and Tmall frequently use recommendation systems [1,2]. Customers receive personalized product recommendations from these systems, which improves user experience and promotes spending. The interactions between users and items constitute a large number of behavior sequences, which are used to better capture user interests. However, capturing accurate preferences from dynamic sequences is challenging because it should not only reflect users' long-term interests but also their changing interests [3]. Therefore, **the focus of sequential recommendation research is to successfully capture short-term changes in user preferences and their effects on the initial model embedding.**

<sup>☆</sup> This study received support from the following funding projects: Natural Science Foundation of Xinjiang Uygur Autonomous Region of China (Grant No. 2023D01C17, 2023D01A123, 2022D01C692); National Natural Science Foundation of China (62262064, 61862060); Xinjiang Uygur Autonomous Region Science and Technology Plan Project Tianshan Innovation Team Plan (Grant No. 2023D4012).

\* Corresponding author.

E-mail address: [yangxy@xju.edu.cn](mailto:yangxy@xju.edu.cn) (X. Yang).

<https://doi.org/10.1016/j.compeleceng.2025.110064>

Received 27 July 2024; Received in revised form 20 December 2024; Accepted 7 January 2025

Available online 15 January 2025

0045-7906/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

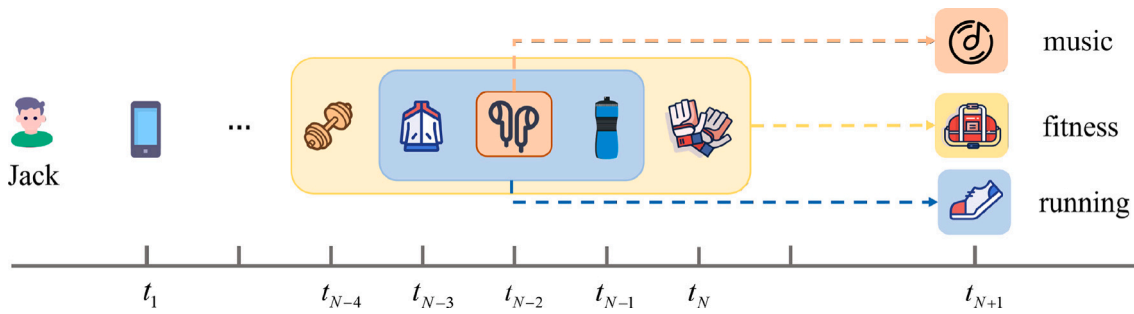


Fig. 1. The impact of different local preference unit sizes on recommendation results.

Recently, the self-attention mechanism has shown its excellent performance in sequential recommendation tasks. Specifically, SASRec [4] is the first sequential recommendation to leverage the self-attention mechanism for capturing patterns of user behavior. By dynamically weighting each step in the user's historical behavior, it precisely identifies key information, thereby better understanding user interests and needs. Given that SASRec only takes into account the user's historical behavior as data, TiSASRec [5] and SSE-PT [6] extend SASRec with additional time interval information and user regularization, respectively. However, these models assume that users' behaviors are strictly ordered, which means that only previous behaviors can affect later behaviors, which limits the ability of the models to capture the potential performance in user sequences. To address this problem, Sun et al. [7] proposed a Bidirectional Encoder Representations from Transformers for Sequential Recommendation (BERT4Rec). However, the above models often rely too much on the simple sequential nature of user historical behaviors, thus ignoring the multidimensional nature of time. In order to address this issue, Cho et al. [8] proposed a Mixture of Attention Mechanisms with Multi-temporal Embeddings (MEANTIME). It integrates temporal information into the attention mechanism, an approach that effectively enhances the model's understanding of temporal dependencies and allows recommender systems to be more accurate in predicting user preferences. Although these approaches perform robustly in terms of overall performance, a large number of parameters are involved by stacking layers of self-attention [9], and the model may suffer from data noise [10] when fitting the data, and may even process data containing malicious forgeries [11]. Consequently, this data often results in overfitting [12], adversely impacting recommendation performance. To determine which type of data affects the model, Zhou et al. [13] conducted denoising experiments on item embeddings using three classical filtering algorithms. The experimental results show that applying low-pass filters (LPF) and band-stop filters (BSF) improves model performance, while high-pass filters (HPF) degrade performance in most cases. Based on these findings, it can be concluded that noise in the data often manifests as high-frequency signals, while low-frequency signals typically contain key information relevant to sequential recommendations. Therefore, further reducing high-frequency noise is crucial for improving recommendation performance.

On the other hand, the aforementioned studies seldom take into account the appropriateness of the scope of user local dependencies, which is essential for capturing the most suitable local preferences. Consider the example of Jack in Fig. 1. When Jack's local preference unit size is 1, we can recommend the "music" option to him based on the local information "earphones". If the preference unit size increases to 3, the additional context items "tracksuit" and "water bottle" suggest that running is the user's current area of attention, so "running shoes" could be a better advice. Further increasing the preference unit size to 5, the additional context items "dumbbells" and "gym gloves" indicate that Jack is likely going to the gym, so a "gym bag" might be the most suitable suggestion. The aforementioned examples demonstrate how various local preference units might yield recommendation results that are richer and more thorough. Thus, the challenge of modeling customers' heterogeneous local preferences persists.

Meanwhile, in conventional self-attention mechanisms, item embeddings and item position embeddings are simply added together. This approach means that each item's representation includes both its intrinsic features and its positional features within the sequence. However, recent studies [14] have indicated that there is a weak correlation between items and their absolute positions. This straightforward addition can degrade model performance, as absolute positional information may not accurately represent the actual relationships between items and can impede the model's ability to capture other critical information in the sequence data. Consequently, traditional methods may not be optimal, necessitating more innovative techniques to better manage the relationship between item and positional features.

To tackle these challenges, we propose a Locally Enhanced Denoising Self-Attention Networks and Decoupled Position Encoding for Sequential Recommendation (LEDADP). It not only reduces the noise in the data, but also makes up for the lack of capturing local dependencies. At the same time, it separates item embedding and item position embeddings, which is more conducive to user sequence modeling. Specifically, we first propose a noise filtering module (NFM) that translates the input representation into the frequency domain by performing a Fast Fourier transform (FFT) [15], optimizes the original data in the frequency domain, and eventually recovers the denoised representation through an inverse Fourier transform process. Our NFM effectively filters out irrelevant or interfering information and extracts significant characteristics from various frequency ranges. Secondly, we develop a multi-scale local enhanced convolution module (MLEC) that applies deep convolution to more accurately capture the local features of each item and its surrounding neighbors. Then, we divide the convolutional channels from small to large according to their sizes to capture the local weights corresponding to different channel sizes. Additionally, we also use a multi-scale correlation modeling

mechanism to enable it to fuse local information from multiple channels. This allows us to discover local representations of greater quality. Finally, we adopt a decoupled position encoding strategy, which is independent of item encoding and can calculate position correlation separately, which explicitly models the sequential relationship between items, making it able to capture users' sequential patterns more effectively. In summary, the main contributions of this paper are as follows:

- A noise filtering module was developed to transform item embeddings from the time domain to the frequency domain, where data optimization is conducted. In this domain, noise is filtered out from the data, facilitating the more effective extraction of meaningful features
- A multi-scale local enhanced convolutional module was proposed to account for varying local preferences across multiple scales, thereby capturing richer local semantic information
- Decoupled position encoding was employed to calculate positional correlations independently, thereby enabling more effective capture of the sequential relationships between items.
- We demonstrated the efficacy of LEDADP by conducting comprehensive experiments on three practical datasets: Beauty, Toys, and ML-1M.

## 2. Related work

### 2.1. Sequential recommendation

Sequential recommendations provide personalized suggestions by analyzing the user's past behavioral data and item characteristics in order to recommend highly relevant content to the user. Early sequential recommendation solutions were mainly based on Markov Chain (MC) [16], where a transfer matrix was generated by calculating the transfer probabilities between each item and its neighbors to make recommendations. However, since MC only consider relationships between neighboring items, it can only capture local sequential patterns, ignoring long-distance dependencies and more complex behavioral patterns. Rendle et al. [17] developed the FPMC model, which combines the benefits of Matrix Factorization (MF) and MC methods, enabling FPMC to not only take into account the historical interaction records of users, but also make full use of transfer relationships between items. Later, neural network-based models began to emerge. Recurrent neural networks (RNNs) have shown outstanding performance in sequence modeling and memory mechanism tasks [18], improving recommendation performance [19]. Convolutional neural networks (CNNs) perform convolutional operations on input sequences through a sliding window mechanism, which can identify and capture features at different levels and effectively extract local patterns and short-term dependencies [20]. The emergence of Transformer has inspired the exploration of its application to sequential recommendation tasks. SASRec calculates the importance of each item in a sequence of user behaviors by means of a self-attention mechanism. BERT4Rec enhances the ability to capture sequence dependencies by means of a bidirectional self-attention mechanism, which also takes into account the contextual information of the items in the sequence. These two approaches are successful representatives of applying Transformer in sequential recommendation tasks. Subsequently, some researchers considered integrating frequency domain features and capturing multi-scale preferences to enhance recommendation performance. Zhang et al. [21] proposed an attention-based frequency-aware multiscale network, which achieves accurate prediction of user intent by transforming user behavior sequences into the frequency domain to generate multi-frequency embeddings, and then focusing on the important frequency components using a frequency-aware attention layer, and combining with a non-local attention module for aggregation on attribute-level and item-level features. Du et al. [22] proposed a Frequency Enhanced Hybrid Attention Network, which is capable of capturing both high-frequency and low-frequency information about user behavior by integrating self-attention mechanisms in both frequency and time domains to achieve accurate modeling of periodic features of sequential data. Meanwhile, MLP can learn visual representations [23] by stacking several MLP layers, giving them performance that can compete with other models. Where the traditional MLP-Mixer [24] focuses on data processing in a two-dimensional space, focusing on the relationships between items. MLP4Rec [25], on the other hand, innovates on this basis by adding a third dimension, enabling the model to handle more complex input data structures. However, most of the above models ignore the noise data hidden in the user's historical interaction sequence.

### 2.2. Local-global awareness

Local-global awareness is a widely discussed issue in the field of recommender systems. Prior research wanted to improve recommendation effectiveness by combining local and global information [26]. However, these approaches usually simply connect different neural modules, and this simple connection fails to take full advantage of the complex dependencies in the data. There is also a lack of adaptive and learnable preference parameters, resulting in models that underperform in capturing dynamic user preference and interest changes. Xu et al. [27] presented a Recurrent Convolutional Neural Network (RCNN) that combines the advantages of RNNs and CNNs. Through this combination, RCNN achieves the dual function of local and global perception, capturing both short-term changes in users' interests and comprehensively understanding their long-term preferences. Yang et al. [28] proposed LESA approach divides self-attention into two components: one part captures global context information, while the other enhances local features through convolution operations. Liu et al. [29] adopted a non-parametric mixture Gaussian distribution with multiple components to capture the variety of user preferences. A local generative model is generated through automatic correlation analysis and later combined with global prediction. Both short-term interests and long-term preferences of users are also taken into account. Some other works have also considered adaptive hybrid of local-global representation learning and showed some advantages,

for example, He et al. [30] used parameterized adaptive predictors to produce highly adaptable mask scores that integrate local boundaries with overall information. Jiang et al. [31] presented an innovative approach to capture user behavioral patterns more accurately by generating importance parameters that dynamically adjust the weight of each local feature in the global sequence. Nevertheless, a lot of current models often overlook the size of the local preference range. In contrast, our model adopts the MLEC module to fuse preferences at multiple scales to obtain richer local representations.

### 2.3. Attention mechanism

The Attention Mechanism (AM) improves the precision of recommendations by dynamically modifying the weights of the input features so that the model can focus more precisely on the key information [32]. It demonstrates its strong superiority in sequential recommendation tasks. Li et al. [33] presented a model called NARM, which combines AM and RNNs specifically designed to capture the user's primary purpose in the current session. Zhou et al. [34] presented a method for processing heterogeneous user inputs using an AM. Subsequently, the appearance of the self-attention mechanism attracted attention. The central idea of the self-attention mechanism is to measure the correlation between items by calculating their attention scores, while the multi-attention mechanism allows the model to capture features separately from different potential subspaces. In other fields, Li et al. [35] proposed a method called SACNN, in which convolutional neural networks are combined with self-attention mechanisms to further improve the sharpness of low-dose CT images. Ding et al. [36] proposed a generative adversarial network based on self-attention, called SAGAN. Convolution and self-attention mechanisms are also combined in the model. The convolution layer is used to extract local features of seismic signals, and the self-attention layer is used to enhance the global consistency of these local features, so as to generate denoised signals with higher structural fidelity. In the field of sequential recommendation, Kang et al. [4] for the first time introduced the self-attention mechanism into sequential recommendation and proposed the SASRec model, which significantly improves the efficiency of training by calculating the importance of each item in a sequence of user behaviors through the self-attention mechanism. Cai et al. [37] presented a category-aware collaborative sequential recommendation approach. It uses self-attention to capture switching patterns within categories, while item categories are used to retrieve users with similar category preferences to enhance dependency modeling and collaborative learning to overcome the data sparsity problem. Chen et al. [38] introduced TLSRec, which leverages a hierarchical self-attentive network to detect user interests on both local and global time scales. In order to dynamically adjust the weights of long-term and short-term preferences, TLSRec also introduces a neural time-gate mechanism, which allows the model to flexibly respond to changes in user interests. However, many self-attention models simply add item embeddings and item position embeddings, which limits the model's ability to capture the user's sequential patterns.

## 3. Method

### 3.1. Problem statement

Before introducing the proposed approach, it is necessary to declare a set of symbols in this paper. Suppose we have a set of users and items, denoted by  $U$  and  $I$  respectively, where  $U = \{u_1, u_2, \dots, u_{|U|}\}$  represents the set of users and  $I = \{i_1, i_2, \dots, i_{|I|}\}$  represents the set of items.  $|U|$  and  $|I|$  represent the number of users and items respectively. Generally speaking, user and item interactions are arranged in chronological order, denoted as  $S_N^u = \{i_1^u, i_2^u, \dots, i_N^u\}$ , where  $i_t^u$  is the item with which user  $u$  interacted at time step  $t$ . The goal of sequential recommendation is to predict the next item, which is  $i_{N+1}^u$ .

### 3.2. Model architecture

The architecture of the LEDADP model is shown in Fig. 2. First, the sequence of  $N$  items passes through an embedding layer, and in the process transforms the original item data into a vector representation in  $d$  dimensions, thus generating the input matrix  $E \in \mathbb{R}^{N \times d}$ . The subsequently generated item embedding matrix  $E_u \in \mathbb{R}^{N \times d}$  is then sent to the NFM, which uses FFT to transform the input representation to the frequency domain. After that, we encode the filter with learnable parameters into the input sequence in the frequency domain to achieve noise reduction, and finally restore the input representation to the time domain through inverse FFT. The MLEC module takes into account the local features of each item and its surrounding neighbors in the sequence. In addition, it also integrates preferences at multiple scales to obtain richer local representations. The generated local representations are subsequently added to the item position embeddings from the position embedding layer to form the final embeddings. This approach explicitly specifies sequence relationships without being affected by the correlation between items and positions, thereby enhancing the model's representation quality. Finally, cross-entropy loss is employed to calculate the matching probabilities.

### 3.3. Embedding layer

The embedding layer will be divided into two later, namely the item embedding layer and the position embedding layer. The item embedding matrix is denoted as  $E_u = [e_1^u, e_2^u, \dots, e_N^u] \in \mathbb{R}^{N \times d}$ , where  $e_i^u$  represents the latent dimension, encoding each scalar item ID  $i_i^u \in \mathbb{R}^{N \times d}$  into a dense latent vector representation  $e_i^u \in \mathbb{R}^{N \times d}$ , aiming to reduce the amount of computation and improve representation learning. The item position embedding is represented as  $P = [p_1, p_2, \dots, p_N] \in \mathbb{R}^{N \times d}$ . Different from the general self-attention mechanism, the model only generates item position embedding in the embedding layer without directly fusing it with the item embedding.

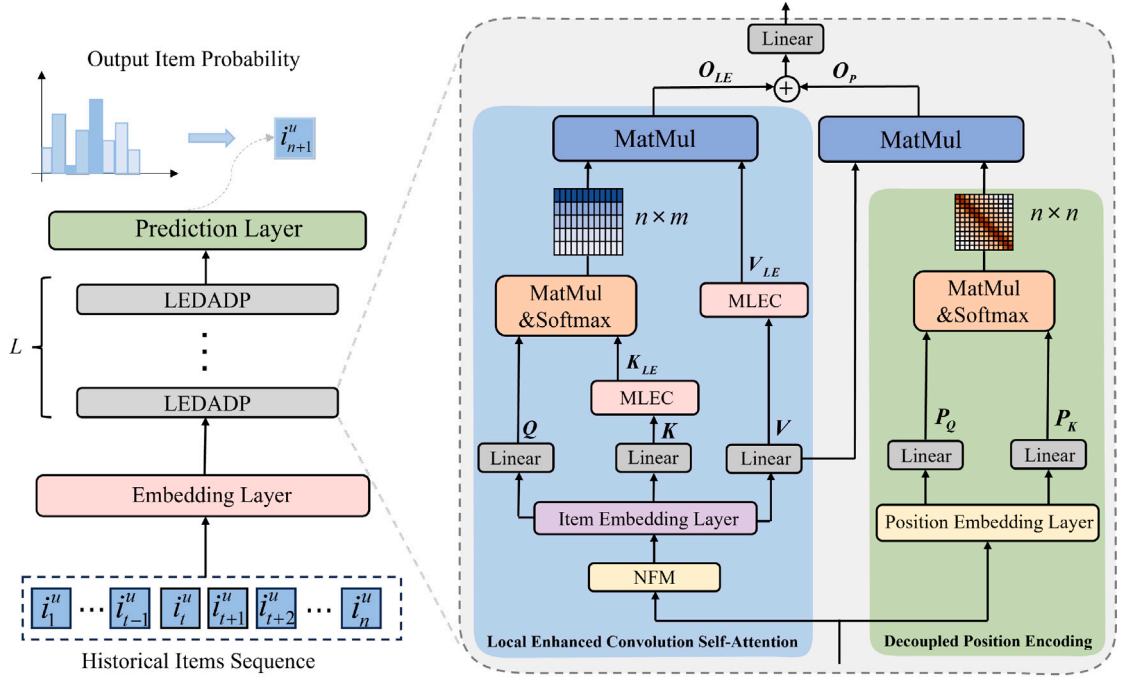


Fig. 2. LEDADP overall framework diagram. LEDADP first generates the input matrix through the embedding layer, and subsequently the generated item embedding matrix is processed through the Noise Filtering Module (NFM) for noise reduction. Then, it is locally enhanced by the Multi-scale Locally Enhanced Convolution Module (MLEC). Afterwards, the processed item embedding is summed with the decoupled positional embedding to form the final embedding. Finally, the cross-entropy loss is utilized for calculating the matching probability.

### 3.4. Noise filtering module

FFT is an efficient algorithm widely used in the field of signal processing by transforming a signal from the time domain to the frequency domain [39]. In the frequency domain, noise is identified by analyzing the amplitude and phase of the spectrum. Once the noise components are identified, they can be filtered out from these frequency bands. For example for removing high frequency noise, a low pass filter is used which allows low frequency signals to pass while suppressing high frequency noise [40]. The noise filtering module in this section uses the same idea.

To attenuate the noise that may exist in the original data, we develop an item encoder based on the item embedding layer, which stacks multiple filters with learnable parameters. In this module, we perform filtering operations on each dimension of the features in the frequency domain, followed by skip connections. Given the input representation  $F^L \in \mathbb{R}^{N \times d}$  of the item at the  $L$ -th layer. When  $L = 0$ , setting  $F^0 = E_u$ , we first perform an FFT on the item dimension to convert  $F^L$  into the frequency domain:

$$X^L = f(F^L) \in \mathbb{C}^{N \times d} \quad (1)$$

where  $f(\cdot)$  represents the one-dimensional FFT. Note that  $X^L$  is a complex tensor representing the spectrum of  $F^L$ . We can then modulate the frequency spectrum by multiplying the learnable filter matrix  $W_f \in \mathbb{C}^{N \times d}$ :

$$\tilde{X}^L = W_f \odot X^L \quad (2)$$

where  $\odot$  represents element-wise multiplication. Filter  $G$  is known as a trainable filter. It continuously adjusts its parameters during the training process to optimize its feature extraction capability on the input data. In this way,  $G$  is able to accurately identify and extract important features related to sequences in the frequency domain, thus improving the performance and accuracy of the model. Ultimately, the sequence representation is updated and the modulated frequency spectrum  $\tilde{X}^L$  is transformed to the temporal domain using the inverse FFT.

$$\tilde{F}^L \leftarrow f^{-1}(\tilde{X}^L) \in \mathbb{R}^{N \times d} \quad (3)$$

Where  $f^{-1}(\cdot)$  represents the one-dimensional inverse FFT, which converts the complex tensor into a real tensor. The FFT and inverse FFT processing effectively reduces the noise from user behavior data, thereby producing purer item embeddings. Our NFM is placed as a component before the item embedding layer. This design allows the NFM to intervene at the data input stage, which effectively reduces the reliance on noisy data and the problem of overfitting. It not only optimizes the data quality, but also improves the stability of the model in the face of complex data environments. Fig. 3 displays the NFM's detailed structure.

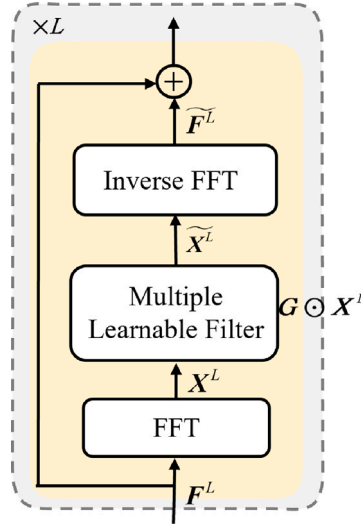


Fig. 3. The overall architecture of NFM.

### 3.5. Multi-scale local enhancement convolution module

The traditional self-attention mechanism usually transforms the embedding matrix  $E$  into  $O_{item}$  after the following operation:

$$\begin{aligned} O_{item} &= \text{softmax} \left[ \frac{EW_Q(EW_K)^T}{\sqrt{d}} \right] EW_V \\ &= \text{softmax} \left( \frac{QK^T}{\sqrt{d}} \right) V \end{aligned} \quad (4)$$

The three linear transformation matrices, denoted by  $W_Q$ ,  $W_K$ ,  $W_V \in \mathbb{R}^{d \times d}$ , transfer the input data to various feature spaces. The scaling factor  $\sqrt{d}$  is responsible for efficiently regulating the attention weights' range. As in common terms  $Q$ ,  $K$  and  $V \in \mathbb{R}^{d \times d}$  denote “query”, “key” and “value” respectively. Conventional self-attentive networks effectively model global information by aggregating the representation of each item to extract context-aware content. This approach captures global patterns and relationships in the data well, but is deficient in capturing local representations. Therefore, in order to obtain more comprehensive local semantic information, a MLEC is inserted after computing the “key” and “value” respectively.

This module primarily comprises of a neighborhood information aggregation module combined with a multi-scale correlation modeling mechanism. The neighborhood information aggregation module captures local adjacent information of items by performing convolution operations and activation function processing on the input features; the multi-scale correlation modeling mechanism uses convolution kernels of different scales to model and process multi-level feature information in the input data. CNNs play a key role in this process. CNNs are able to efficiently extract local patterns and dependencies in sequences, reduce the number of model parameters and improve the computational efficiency through the mechanism of local sensory fields and weight sharing. Specifically, the convolution operation automatically learns local correlations and patterns in the sequence by sliding multiple convolution kernels over the input sequence. The size of these convolutional kernels determines the length of the receptive field, which affects how finely the model captures features. In our MLEC module, the local information of different sizes of convolutional kernels is fused, and different sizes of convolutional kernels determine different lengths of local receptive fields, which enables the model to acquire feature information at multiple scales simultaneously, and thus capture the features of sequences at multiple scales. The structure diagram of the MLEC module is shown in Fig. 4.

#### 3.5.1. Neighborhood information aggregation module

The module is made up of an input channel embedding, an activation function (GELU), a local enhanced convolution module (LEC), and an output channel embedding. Specifically, given the input matrix  $A$ , the following formula is used to determine the output:

$$Y = \text{GELU}(\text{LayerNorm}(A)W_1 + b_1) \quad (5)$$

$$A_{LE} = A + \text{LEC}(Y)W_2 + b_2 \quad (6)$$

Where  $W_1 \in \mathbb{R}^{d \times Y_{LB}}$  and  $W_2 \in \mathbb{R}^{Y_{LE}/2 \times d}$  represent two channel embeddings, which are used to adjust the dimensional relationship between input features and hidden representations to enhance the learning ability of the model.  $b_1 \in \mathbb{R}^{Y_{LE}}$ ,  $b_2 \in \mathbb{R}^d$  denotes the vector used for bias.  $Y_{LE}$  is the hidden dimension used to capture the richer underlying semantics. In this case,  $Y_{LE} = 4d$  is set.



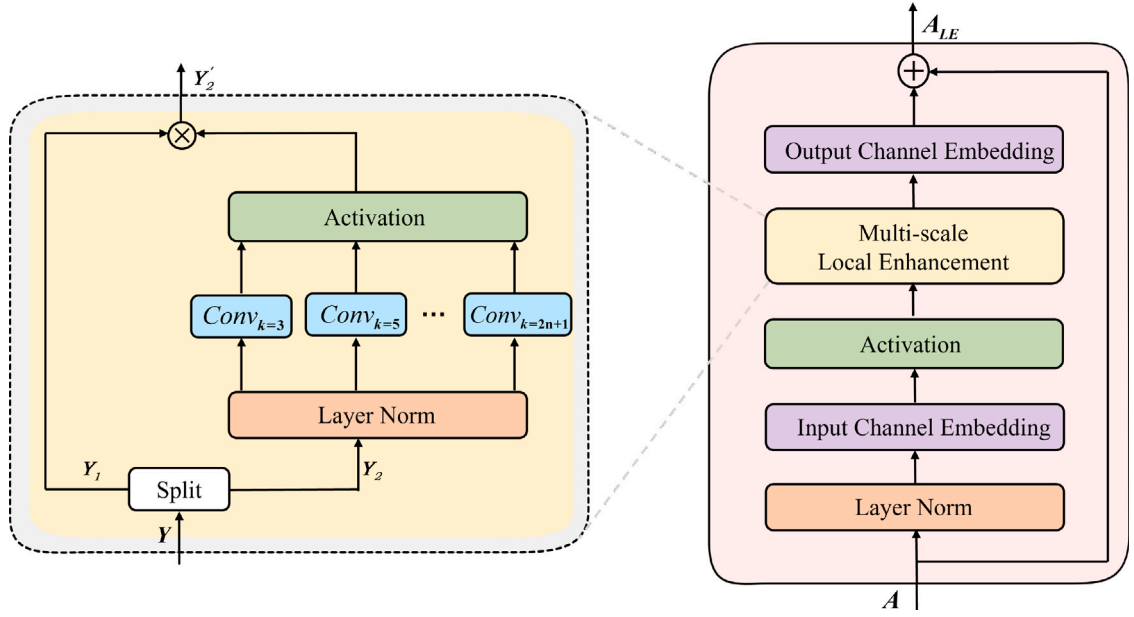


Fig. 4. The overall architecture of MLEC Module.

The LEC module enhances the representation of items by aggregating neighboring information. To be more precise, it first splits the input sequence  $Y \in \mathbb{R}^{N \times Y_{LE}}$  into two parts based on the hidden dimensions:  $Y_1$  and  $Y_2 \in \mathbb{R}^{N \times Y_{LE}/2}$ . In order to guarantee the consistency and stability of the data on the same scale,  $Y_2$  is first normalized. Then, to finely capture local dependencies, deep convolutional processing is applied to the normalized  $Y_2$ . The link between the items and the nearby data points is strengthened by this processing, which also extracts more detailed local features. In particular, the following is how LEC calculates its ultimate output:

$$Y_2' = \text{GELU}(\text{DWConv}(\text{LayerNorm}(Y_2))) \quad (7)$$

$$\text{LEC}(Y) = Y_1 \odot Y_2' \quad (8)$$

Where DWConv [41] denotes deep convolution, which belongs to a type of convolution operation in CNNs, it performs spatial convolution operations on each input channel individually, thus significantly reducing the number of parameters and computational complexity. In this way, the features of each channel can be processed independently, which not only improves the computational efficiency but also reduces the computational resource consumption of the model.  $Y_2' \in \mathbb{R}^{N \times Y_{LE}/2}$  denotes the locally enhanced convolution weights. In this context, the GELU function is employed as the activation function to enhance the model's fitting and expressive capabilities. Through element-wise multiplication of  $Y_1$  and  $Y_2'$ , a locally enhanced sequence representation is generated.

### 3.5.2. Multi-scale correlation modeling mechanism

DWConv is essential for capturing local information, however, the optimal configuration for the convolution kernel  $k$  remains a matter of debate. Previous studies typically set this as a manually adjusted hyperparameter, but this approach is viewed as lacking adaptability. Therefore, we propose a multi-scale correlation modeling mechanism. When modeling local dependencies, the mechanism can take into account local preferences at different scales, thus offering richer and more useful adjacent information for key items.

Specifically, convolution kernels of varying sizes  $k_1, k_2, \dots, k_n (k_1 = 3, k_2 = 5, \dots, k_n = 2n + 1)$  are employed to obtain the detailed features of first-rank, second-rank, ..., and  $n$ th rank neighbors, separately. The features extracted from these convolution kernels at different scales are multiplied and fused to form a weight matrix containing richer and more comprehensive local information. This approach not only captures the neighbor relationship at different scales, but also enhances the comprehensive representation of local features, thus improving the performance of the model when modeling multi-scale local information. Mapping to the equation, we rewrite the computation of  $Y_2'$  in Eq. (7) in the following form:

$$Y_2' = \prod_{k=1}^n \text{GELU}(\text{DWConv}_{2k+1}(\text{LayerNorm}(Y_2))) \quad (9)$$

where  $n$  is the upper limit of the scale, often not exceeding half the length  $N$  of the sequence and  $2k + 1$  represents the kernel size of the DWConv. Furthermore, an element-wise product is employed rather than direct summation to integrate the multi-scale local weights, as it effectively avoids the issue of weight imbalance between different scales. In additive fusion, the influence of certain

scales may be overly emphasized, leading to the loss of information. In contrast, element-wise multiplication allows each scale's local information to maintain a certain degree of independence and relative importance during integration, which helps preserve the balanced contribution of each scale's features in the final output.

In summary, our multi-scale correlation modeling mechanism, by introducing convolution kernels of different scales and an element-wise multiplication fusion strategy, not only enhances the ability to model local features but also effectively avoids the issue of weight imbalance between scales, thereby improving the overall performance and adaptability of the model.

Thus, after processing through the multi-scale local enhancement convolution module, the inputs  $K$  and  $V$  are transformed into  $K_{LE}$  and  $V_{LE}$  respectively, with the corresponding perceptual context output expressed as:

$$O_{LE} = \text{softmax} \left( \frac{QK_{LE}^T}{\sqrt{d}} \right) V_{LE} \quad (10)$$

### 3.6. Decoupled position encoding

Earlier attention-based models commonly combine item embeddings ( $E$ ) and item position embeddings ( $P$ ) to establish sequential relationships, represented as  $(E + P)$ . Therefore, the correlation between two items  $i$  and  $j$  is denoted as  $(E_i + P_i)(E_j + P_j)^T$ . However, this approach yields a weak correlation between items and positions, which limits the model's ability to capture sequential relationships within the sequence. Consequently, in this work, we employ a decoupled position encoding to model the sequential relationships between items, allowing the position encoding to remain independent from context-aware representation modeling:

$$\begin{aligned} O_{\text{item}} &= \text{softmax} \left[ \frac{PZ_Q(PZ_K)^T}{\sqrt{d}} \right] V \\ &= \text{softmax} \left( \frac{P_Q P_K^T}{\sqrt{d}} \right) V \end{aligned} \quad (11)$$

Among them,  $Z_Q$  and  $Z_K \in \mathbb{R}^{d \times d}$  are learnable parameters. The computations of the above equations allow  $O_{LE}$  and  $O_p$  to be calculated independently and in parallel. Furthermore, the sequential relationships are explicitly defined, remaining unaffected by the correlation between items and positions, thereby enhancing the performance of our model.

It is also noteworthy that since  $O_p$  is independent of the input, it can be computed within the same batch and shared among all users. The computational cost of  $O_p$  is negligible during both the training and testing phases.

### 3.7. Prediction layer and loss function

At last, we integrate the output obtained from the MLEC module with the item position embeddings to get the final output  $O = g[(O_{LE} + O_p)H]$ , where  $H \in \mathbb{R}^{d \times d}$  represents a linear projection, and  $g$  denotes a mapping that is formed after linear transformation and nonlinear activation processing.

We encode the input sequence using  $L$  stacked LEDADP layers to generate the output  $O^L$ . The final item  $i_N^u$  in the input sequence  $\{i_1^u, i_2^u, \dots, i_N^u\}$  is predicted as  $O_N^L$ . Our model is trained end-to-end by minimizing the cross-entropy loss:

$$\mathcal{L}(\Theta) := -\log \frac{\exp(\langle O_N^L, E_{i_N^u} \rangle)}{\sum_{t=1}^{|I|} \exp(\langle O_N^L, E_t \rangle)} \quad (12)$$

Where  $\langle O_N^L, E_t \rangle$  represents the dot product between  $O_N^L$  and any item  $i_t$ , while  $\Theta$  represents the parameters that can be trained, including some matrices, vectors and network parameters.

## 4. Experiments

### 4.1. Experiment setup

In this section, the dataset, evaluation metrics, baseline models, and experimental setup are first described. A comprehensive evaluation is then conducted on the experimental results.

#### 4.1.1. Dataset

We carried out comprehensive tests on three publicly available datasets with different domains and varying data sparsity. These experiments aim at evaluating the performance of our approach under different data characteristics and application scenarios.

- (1) Amazon Beauty and Toys [42]: The dataset contains comments and ratings for a variety of items grabbed from Amazon. It is separated into various categories, with the "Beauty" and "Toys" categories being used in this paper.
- (2) MovieLens-1M [43]: The MovieLens dataset, which consists of users' historical movie ratings, serves as a benchmark for recommender system research. For the experiments in this paper, we choose the MovieLens-1M version, which contains a large amount of detailed user rating data for movies and is suitable for evaluating the performance of recommender systems.



**Table 1**  
Statistics of the preprocessed dataset.

| Dataset | #Users | #Items | #Actions | #Len <sub>U</sub> | #Len <sub>I</sub> | #Sparsity |
|---------|--------|--------|----------|-------------------|-------------------|-----------|
| Beauty  | 22 363 | 12 101 | 198 502  | 8.9               | 16.4              | 99.93%    |
| Toys    | 19 412 | 11 924 | 167 597  | 8.6               | 14.1              | 99.93%    |
| ML-1M   | 6041   | 3417   | 999 611  | 165.4             | 292.6             | 95.15%    |

These datasets are the ones used in many previous works, and we follow the same data preprocessing procedures. Essentially, we transform each dataset into an implicit dataset, treating each rating or review as the presence of a user–item interaction. Subsequently, interactions are grouped by user ID and sorted by timestamps to create sequences for each user. Table 1 presents the dataset statistics, where #Len<sub>U</sub> and #Len<sub>I</sub> indicate the average number of user and item interactions, respectively. These three datasets are frequently employed as public benchmarks, differing in size and density, thereby providing an opportunity to evaluate the model across diverse types of datasets. The aforementioned datasets can be downloaded from the provided link.<sup>1</sup>

#### 4.1.2. Evaluation metrics

To evaluate the performance of each method, we adopted the widely used leave-one-out method. The dataset was divided into training, validation, and test sets based on the timestamps. Specifically, the last item was used for testing, the second-to-last item for validation, and the remaining items for training. Additionally, following the common practice described in [13,31], each actual item in the test set is paired with 100 randomly sampled negative items, none of which have been interacted with by the user. To enhance the realism of the task, we extract negative items by uniform sampling. The ranking lists were evaluated using two common metrics: Recall and Normalized Discounted Cumulative Gain (NDCG). When the actual items rank high in the top- $K$  lists, both Recall@ $K$  and NDCG@ $K$  values are significant. We record the Recall and NDCG values for  $K = 5, 10$ .

#### 4.1.3. Baseline models

We selected some typical state-of-the-art model methods for comparison, including general recommendation methods (GRU4Rec, NARM, NextItNet, SASRec), self-attention based sequential recommendation methods (LightSANS, PAUP), time–frequency domain information transformation based recommendation methods (FMLP-Rec, FEARec), and recommendation methods combine global and local dependencies (AdaMCT, TriMLP, MLM4Rec). The following provides an overview of the baselines:

- GRU4Rec [44]: A model that employs gated recurrent unit layers to model the sequential behavior of items within sessions for recommendations.
- NARM [33]: A session recommendation model utilizing a hybrid encoder and attention mechanisms.
- NextItNet [45]: A straightforward convolutional generative network aimed at next-item recommendation.
- SASRec [4]: A transformer-based model leveraging self-attention networks for sequential recommendations.
- LightSANS [46]: A self-attention network based on low-rank decomposition for next-item recommendation.
- PAUP [47]: A model employing asymmetric position encoding and a progressive self-attention network for sequential recommendation.
- FMLP-Rec [13]: A fully MLP model designed for sequential recommendation tasks that utilizes learnable filters in the frequency domain to alleviate noise issues.
- FEARec [22]: A hybrid attention mechanism that combines both time domain and frequency domain is used to capture user preferences for sequential recommendation.
- AdaMCT [31]: An adaptive hybrid CNNs transformer designed for sequential recommendation.
- TriMLP [48]: A framework based on MLP that employs a triangular mixer with “Global Mixing Layer” and “Local Mixing Layer” for sequential recommendation.
- MLM4Rec [49]: A sequential recommendation that uses an MLP architecture to simultaneously capture global and local dependencies, and combines these two representations through an adaptive fusion layer.

#### 4.1.4. Implementation details

We evaluate the baseline models based on the RecBole framework [50,51] and conduct experiments using the optimal hyperparameter settings specified in the original papers. LEDADP is implemented based on the PyTorch framework. For a fair comparison, the same hyperparameter settings are used as those of the baseline models: The hidden dimension of the embeddings  $d$  is set to 64, the batch size is fixed at 1024, Adam is used as the optimizer, and the learning rate is set to 0.001. For the self-attention network, the number of attention heads is fixed at 2, and the number of network layers is also fixed at 2. Due to the different sparsity of datasets, we set the hidden dropout rate to 0.2 for ML-1M and 0.5 for other datasets. The maximum sequence lengths  $N$  are configured as 50 and 150, applied to two Amazon datasets and ML-1M respectively. The maximum convolution kernel sizes  $k$  are selected from 3, 5, 7, 9 and the number of layers  $L$  in the noise filtering module is chosen between 1 and 4 with a step size of 1. Initially, each model was trained using an early stopping strategy, ceasing training when the validation accuracy did not improve for 10 consecutive epochs on a single NVIDIA 4090 24 GB GPU.

<sup>1</sup> <https://github.com/RUCAIBox/RecSysDatasets>

Table 2

Performance comparison with the baseline model.

| Methods        | Beauty     |             |          |           | Toys       |             |          |           | ML-1M      |             |          |           |
|----------------|------------|-------------|----------|-----------|------------|-------------|----------|-----------|------------|-------------|----------|-----------|
|                | Recall@5 ↑ | Recall@10 ↑ | NDCG@5 ↑ | NDCG@10 ↑ | Recall@5 ↑ | Recall@10 ↑ | NDCG@5 ↑ | NDCG@10 ↑ | Recall@5 ↑ | Recall@10 ↑ | NDCG@5 ↑ | NDCG@10 ↑ |
| GRU4Rec [44]   | 0.0192     | 0.0297      | 0.0123   | 0.0157    | 0.0190     | 0.0289      | 0.0122   | 0.0153    | 0.1196     | 0.2044      | 0.0765   | 0.1037    |
| NARM [33]      | 0.0223     | 0.0335      | 0.0146   | 0.0180    | 0.0230     | 0.0337      | 0.0148   | 0.0183    | 0.1186     | 0.1948      | 0.0724   | 0.0968    |
| NextItNet [45] | 0.0170     | 0.0269      | 0.0108   | 0.0140    | 0.0100     | 0.0153      | 0.0063   | 0.0080    | 0.0492     | 0.0863      | 0.0305   | 0.0424    |
| SASRec [4]     | 0.0377     | 0.0549      | 0.0223   | 0.0278    | 0.0363     | 0.0523      | 0.0209   | 0.0261    | 0.1254     | 0.2105      | 0.0756   | 0.1029    |
| LightSANS [46] | 0.0413     | 0.0577      | 0.0242   | 0.0296    | 0.0416     | 0.0598      | 0.0239   | 0.0297    | 0.1426     | 0.2264      | 0.0883   | 0.1152    |
| PAUP [47]      | 0.0399     | 0.0580      | 0.0219   | 0.0285    | 0.0404     | 0.0587      | 0.0233   | 0.0293    | 0.1356     | 0.2199      | 0.0828   | 0.1029    |
| FMLP-Rec [13]  | 0.0384     | 0.0567      | 0.0224   | 0.0283    | 0.0385     | 0.0556      | 0.0224   | 0.0279    | 0.1346     | 0.2169      | 0.0826   | 0.1100    |
| FEARec [22]    | 0.0405     | 0.0573      | 0.0238   | 0.0293    | 0.0414     | 0.0597      | 0.0245   | 0.0299    | 0.1417     | 0.2222      | 0.0868   | 0.1143    |
| AdaMCT [31]    | 0.0397     | 0.0546      | 0.0230   | 0.0278    | 0.0401     | 0.0559      | 0.0223   | 0.0274    | 0.1297     | 0.2166      | 0.0814   | 0.1093    |
| TriMLP [48]    | 0.0391     | 0.0574      | 0.0221   | 0.0287    | 0.0406     | 0.0531      | 0.0234   | 0.0289    | 0.1414     | 0.2219      | 0.0854   | 0.1128    |
| MLM4Rec [49]   | 0.0417     | 0.0595      | 0.0246   | 0.0297    | 0.0421     | 0.0604      | 0.0242   | 0.0303    | 0.1437     | 0.2320      | 0.0897   | 0.1193    |
| LEDADP         | 0.0429     | 0.0612      | 0.0251   | 0.0307    | 0.0454     | 0.0626      | 0.0255   | 0.0310    | 0.1496     | 0.2401      | 0.0958   | 0.1251    |

#### 4.2. Overall performance comparison

To validate the effectiveness of our model LEDADP, it was compared with other advanced models. Table 2 presents the performance of both the baseline models and LEDADP across three datasets, where the best performance results are highlighted in bold and the second-best results are underscored. Based on these results, the following analysis was conducted: Among the four general recommendation methods, NextItNet, which relies exclusively on CNNs, has the poorest performance. For the less optimal GRU4Rec and NARM models, NARM performs better on the Beauty and Toys datasets, reflecting the advantages of the self-attention mechanism for sequence modeling. However, on the ML-1M dataset, NARM does not perform as well as GRU4Rec. This is because the ML-1M dataset is characterized by a large number of interactions. GRU4Rec not only captures global dependencies but also effectively captures local dependencies through its gating mechanism, resulting in better performance on the ML-1M dataset. SASRec achieved competitive results by using Transformer, which shows the power of Transformer in sequence modeling. LightSANS and PAUP achieve more effective and precise contextual awareness modeling by integrating low-rank decomposition and down-sampling convolutions into the self-attention mechanism, thus outperforming general recommendation methods. In comparison to Transformer-based methods, LEDADP shows significant improvements. Specifically, relative to SASRec, Recall@5 increases by 13.79%, 25.06%, and 19.2% on the Beauty, Toys, and ML-1M datasets, respectively, while NDCG@5 improves by 12.55%, 22%, and 30.29%, respectively. The reason is that SASRec ignores the noise problem, whereas LEDADP utilizes a NFM to reduce noise, enabling the model to extract more meaningful information.

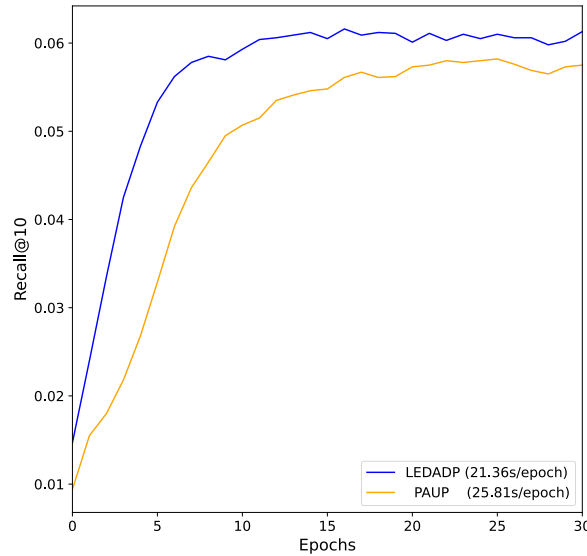
FMLP-Rec, by employing a fully MLP architecture, avoids excessive parameter stacking and incorporates denoising techniques in the frequency domain, enabling the model to effectively capture global user interaction information. As a result, it outperforms general recommendation models. However, this approach also leads to a lack of attention to local information. FEARec improves upon the original time-domain self-attention by enhancing it in the frequency domain, while also designing a frequency-domain attention mechanism based on autocorrelation to capture periodic features. It has achieved better results on certain metrics, indicating that incorporating frequency-domain information is beneficial for improving the model's ability to recognize periodic patterns in user behavior.

The AdaMCT model, which integrates CNNs and Transformers, performs better than standalone CNN and Transformer models. TriMLP not only employs a fully MLP-based architecture but also uses a triangular structure for global and local mixing, demonstrating certain advantages in modeling local dependencies and complex sequential relationships. However, its approach to global and local mixing is relatively simple, which imposes certain limitations on the model's performance. MLM4Rec adopts an MLP architecture to capture global representations in user behavior sequences through an integrated mixer, while enhancing the capture of local dependencies with multi-granularity convolutional operations. It then combines these two representations through an adaptive fusion layer, demonstrating strong performance. The experimental results show that recommendation methods combining global and local dependencies outperform most other baselines, indicating that integrating both global and local dependencies can improve recommendation performance.

Our LEDADP model performs the best on all three datasets, as most of the other models do not take into account different local preferences at multiple scales, which affects the modeling of short-term item interests and ultimately limits their performance. We achieve this by integrating the MLEC module into the self-attention mechanism, allowing the model to dynamically adjust its focus on different local regions when processing various contextual information. This enables our model to consider global dependencies while capturing finer local patterns and trends through convolutional operations, thereby improving the accuracy of its predictions. In conclusion, the experimental results clearly demonstrate that our proposed model, LEDADP, outperforms all baseline models across all datasets and evaluation metrics. For these baselines, we not only solve the problem of not considering local preferences at multiple scales, but also integrate noise filtering modules to reduce noise and employ decoupled position encoding to better represent the sequential relationship of items, so that our model has improvements on all three datasets, further improving the performance of the model.

**Table 3**  
Performance under Different Convolutional Scales.

| Model           | Beauty               |                    | Toys                 |                    | ML-1M                |                    |
|-----------------|----------------------|--------------------|----------------------|--------------------|----------------------|--------------------|
|                 | Recall@10 $\uparrow$ | NDCG@10 $\uparrow$ | Recall@10 $\uparrow$ | NDCG@10 $\uparrow$ | Recall@10 $\uparrow$ | NDCG@10 $\uparrow$ |
| $k = [3]$       | 0.0600               | 0.0289             | 0.0610               | 0.0287             | 0.2342               | 0.1202             |
| $k = [3,5]$     | 0.0605               | 0.0298             | 0.0618               | 0.0297             | 0.2375               | 0.1204             |
| $k = [3,5,7]$   | <b>0.0612</b>        | <b>0.0307</b>      | 0.0621               | 0.0298             | <b>0.2401</b>        | <b>0.1251</b>      |
| $k = [3,5,7,9]$ | 0.0609               | 0.0299             | <b>0.0626</b>        | <b>0.031</b>       | 0.2377               | 0.1238             |



**Fig. 5.** Training efficiency comparison based on Recall@10.

### 4.3. Further analysis

#### 4.3.1. Analysis of convolutional scales

In our proposed MLEC module, the maximum value in the convolution kernel denotes the top bound of the scale. This setting allows our model to focus more on capturing local dependencies, thus enhancing the model's ability to understand and process local information. Experiments were conducted with different convolution scales on three distinct datasets. To evaluate the performance of the model, we set the maximum value of the convolution kernel to 9. Table 3 displays the findings of the experiment.

The table indicates that larger convolution scales produce better performance on the Toys dataset. This implies that fixed-size convolution kernels are less efficient than multi-scale local enhanced convolution at capturing local characteristics. Nevertheless, in our experiments on the Beauty and ML-1M datasets, we observed that a large convolutional kernel does not make the model perform better. While a larger convolutional kernel theoretically captures a wider range of neighborhood information, in practice, this extension may introduce some neighborhood information that is semantically irrelevant to the target item. The accuracy of the local representation may suffer as a result of the model's inability to appropriately collect local features due to the irrelevant neighborhood data.

#### 4.3.2. Efficiency analysis

We used the Beauty dataset to compare our approach with the PAUP model in order to assess its efficiency. We put the data into curves, as seen in Fig. 5 below, based on the experimental findings for each epoch and the model training duration.

It is evident from the graphic that our model starts showing signs of convergence at around 15 epochs, whereas the PAUP model begins this process at approximately 20 epochs. This indicates that our model achieves a faster convergence speed. Furthermore, with respect to training time, the average training time per epoch for our model is 21.36 s, compared to 25.81 s for PAUP, demonstrating higher training efficiency. More importantly, our model also outperforms PAUP in terms of the Recall@10 metric, indicating that our model exhibits certain advantages in both training efficiency and performance.

#### 4.3.3. Visualizing attention weights

In this section, we have visualized the average attention weight values across all test sequences. In particular, in order to preserve the temporal order, SASRec employs a unidirectional attention mask used to ensure that the model can only rely on the previous

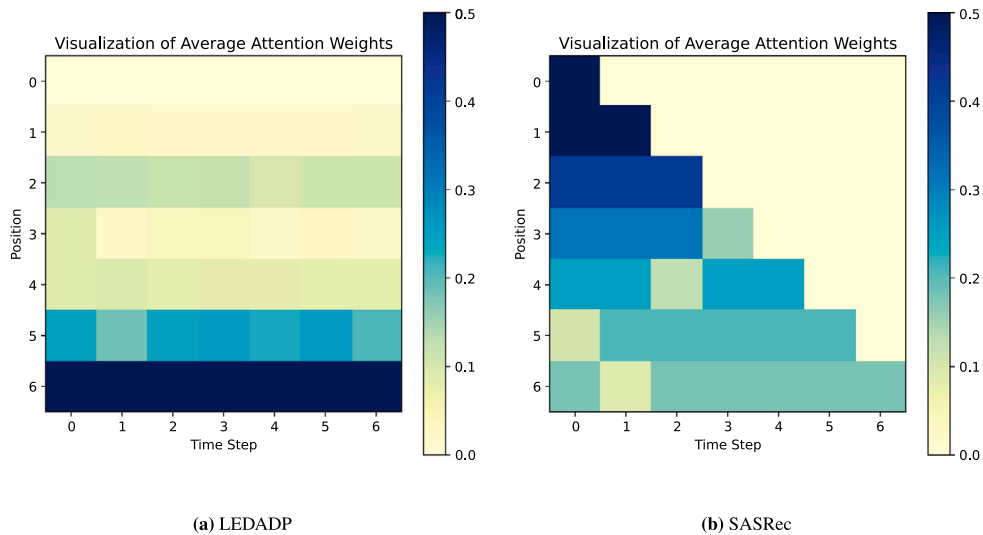


Fig. 6. Visualization of the average attention matrix in the final layer on ML-1M.

behavior of a location when predicting it, and cannot make use of future information, guaranteeing the accuracy of the prediction. Consequently, a triangle-shaped heatmap is produced, as shown in Fig. 6(b). After incorporating the MLEC module in our study, a close examination of Fig. 6(a) reveals that the attention values gradually increase with the position of the subsequence. This trend indicates that newer items have a greater influence when predicting future outcomes. This finding aligns with existing research [52], emphasizing the importance of recent patterns. More importantly, Fig. 6(a) exhibits a unique weight distribution when handling local time intervals, distinguishing it from traditional Transformer models. Specifically, the weight distribution shows the contour of the time window, indicating that our model can more effectively extract valuable information within local time intervals. Our MLEC module, by considering different local preferences at multiple scales, can more effectively capture rich local semantic information, thereby enhancing prediction accuracy and overall model performance.

#### 4.4. Ablation study

In this part, we conduct ablation investigations on a number of important model modules to examine the influence of individual modules on the model's performance and confirm the efficacy of the LEDADP model design. When the NFM, MLEC modules, and decoupled position encoding are removed from the LEDADP model, the resulting variant models are labeled LEDADP-N, LEDADP-M, and LEDADP-D, respectively. The LEDADP model serves as the complete version. Ablation experiments were conducted across three datasets, with the results presented in Fig. 7.

First, we removed NFM. The experimental results demonstrated that LEDADP-N significantly underperformed compared to the full LEDADP model across all three datasets, thereby confirming the critical role of the noise filtering module. This is because, following the removal of the noise filtering module, the model fails to effectively filter noise from the data, leading to increased interference from noisy data during training and prediction. Consequently, the accuracy and robustness of the model are diminished. The NFM is essential for enhancing data quality and improving the purity of feature representation.

Subsequently, the important MLEC module was removed from the model. Fig. 7 shows a decline in the performance of LEDADP-M. The model loses the ability to capture multi-scale local preferences, which hinders its capacity to fully acquire local semantic information at varying scales. Consequently, the model exhibits reduced performance in addressing complex contextual relationships. Additionally, we found that the performance of LEDADP-M on the ML-1M dataset declined noticeably. This is because in sparse datasets, short sequences contain limited local information, making it difficult to fully reflect user preferences. In contrast, the ML-1M dataset has a lot of long sequence data, which provides a more detailed record of user behavior, enabling the model to more accurately understand and predict user interests and needs.

Finally, the removal of the decoupled positional encoding diminished the model's ability to effectively capture item sequence relationships. Because the decoupled positional encoding independently computes positional relevance relative to item embeddings, explicitly modeling the sequential relationships between items. This significantly enhances the model's ability to understand and process sequential data. In addition, the experimental results indicate that, despite the performance decline of all LEDADP variant models following the removal of certain modules, they continue to outperform PAUP. This suggests that the design of each module within the LEDADP model is effective.

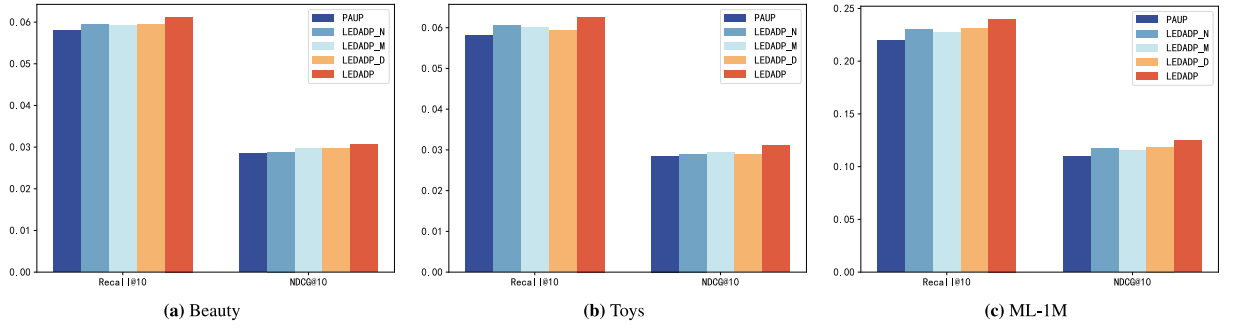
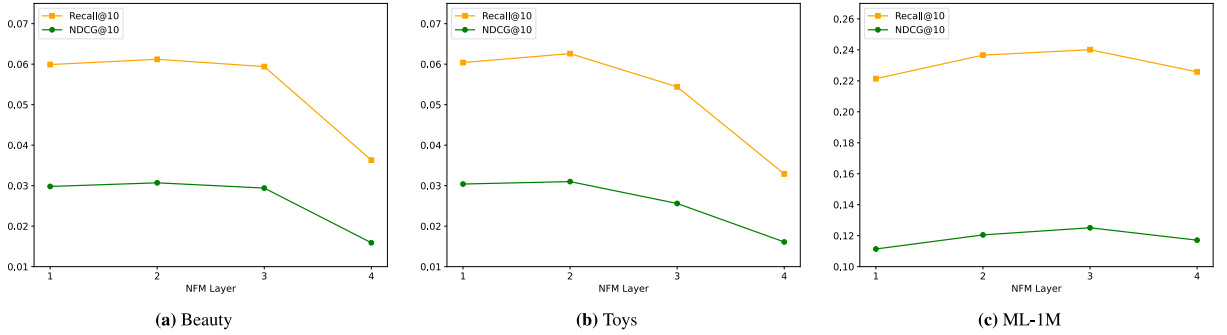


Fig. 7. Ablation experiments on different modules.

Fig. 8. The impact of the number of layers of the noise filtering module  $L$  on model performance.

#### 4.5. Hyper-parameter sensitivity

##### 4.5.1. The impact of the number of layers of the noise filtering module $L$

To examine the impact of the number of layers in the noise filtering module on recommendation performance, we set  $L$  to 1, 2, 3, 4 on the three datasets, with all other parameters configured to their optimal values. The experimental results are presented in Fig. 8.

The findings of the experiment indicate that the model works best when the number of noise filtering module layers  $L$  is set to 2 for the Beauty and Toys datasets, and 3 for the ML-1M dataset. This might be because various datasets have distinct user behavior patterns. For the Beauty and Toys dataset, the user's purchasing behavior is more stable and relatively less noisy compared to the diverse film rating records in ML-1M, so a layer number of 2 is already effective in removing redundant and irrelevant information while retaining key features. In the ML-1M dataset, due to the diversity of user behaviors, deeper layers are required to capture user preferences more accurately. However, with further increase in the number of layers, the performance of the model decreases on all the three datasets, this is because blindly increasing the number of layers may deal with too much noise and thus overfitting phenomenon, leading to a decrease in the performance of the model.

##### 4.5.2. The impact of maximum sequence length $N$

To investigate the impact of maximum sequence length on recommendation performance, we set  $N$  to 10, 20, 30, 40, 50 for the Beauty and Toys datasets, and to 100, 150, 200, 250, 300 for the ML-1M dataset, with all other parameters configured to their optimal values. The experimental results are presented in Fig. 9.

The findings of the experiment show that the optimal parameter for  $N$  is 50 for the Beauty and Toys datasets, while it is 150 for the ML-1M dataset. This suggests that appropriately adjusting the maximum sequence length can significantly improve model performance across different datasets. Further analysis indicates that, with the continuous increase in maximum sequence length, the model's performance gradually declines on the ML-1M dataset. We speculate that, in the ML-1M dataset, the complexity of the data and the diversity of the sequences enable a longer maximum sequence length to better capture useful information. However, exceeding this length may introduce additional noise, which can interfere with the model's ability to capture valuable information, resulting in a decline in performance.

##### 4.5.3. The impact of activation function $\Phi$

To examine the impact of the activation function  $\Phi$  on recommendation performance, the activation functions Sigmoid, TanH, Swish, ReLU, and GELU were selected, with all other parameters maintained at their optimal values. The experimental results are

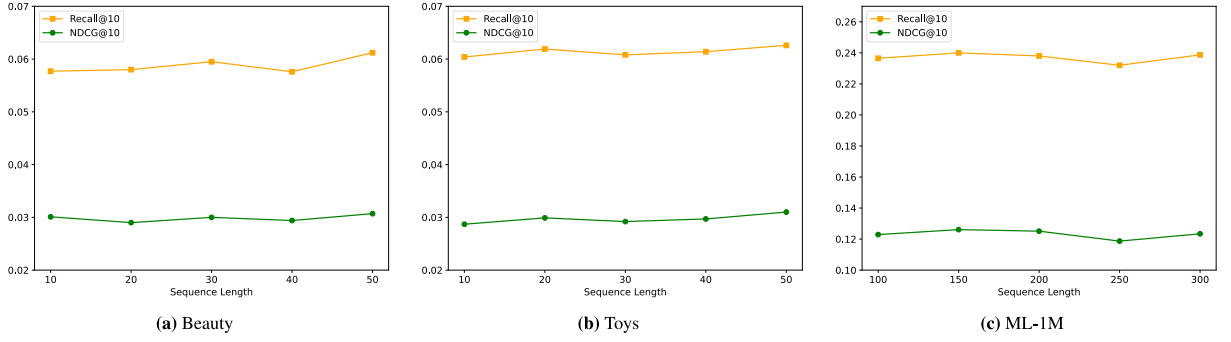


Fig. 9. The impact of maximum sequence length  $N$  on model performance.

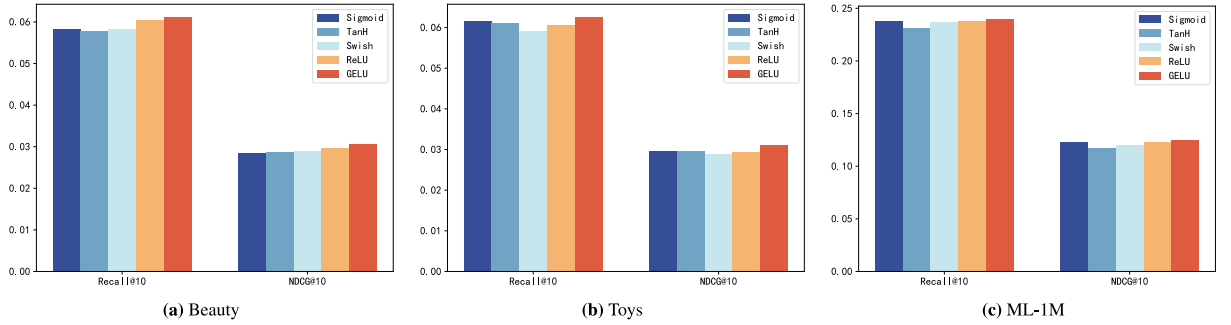


Fig. 10. The impact of activation function  $\Phi$  on model performance.

presented in Fig. 10.

We may conclude from the experimental findings that our model works best when using the GELU activation function. We believe that this is because the NFM allows GELU to process input values more smoothly, retaining more useful information within the signal. Specifically, the GELU activation function processes input data according to a Gaussian distribution through smooth nonlinear transformations. This allows the model to update parameters more stably during training, reducing the instability caused by discontinuities in activation functions. This characteristic perfectly aligns with our NFM, effectively reducing noise interference in the data and enhancing model performance. At the same time, for local interest transitions, the smooth and continuous nature of GELU enables it to better capture subtle changes during convolution operations. Specifically, GELU's smooth curve enables the capture of subtle changes, thereby enhancing the model's accuracy in handling user interest transitions.

## 5. Conclusion

In this study, we suggested a unique Locally Enhanced Denoising Self-Attention Networks and Decoupled Position Encoding for Sequential Recommendation (LEDADP). Specifically, we reduce the noise present in the data through the noise filtering module. Subsequently, a multi-scale local enhancement convolution module was proposed that employs deep convolution to aggregate the neighborhood information of items. This module also incorporates a multi-scale correlation modeling mechanism, which allows for the consideration of local information at various scales, thereby effectively enhancing local feature representation. Finally, decoupled positional encoding was utilized to explicitly model the sequential relationships between items. Compared with the suboptimal method, our method improves the Recall@5 index of the three datasets by 2.87%, 7.83%, 4.1% and the NDCG@5 index by 2.03%, 4.08%, 6.8%, respectively, verifying the effectiveness of the proposed method. In future studies, we will further explore how to adaptively adjust the convolution kernel size according to the characteristics of the input data, that is, the method of dynamically adjusting the convolution kernel to more flexibly capture local information of different scales, thereby improving the recommendation performance.

## CRedit authorship contribution statement

**Xingyao Yang:** Methodology, Writing – review & editing, Funding acquisition, Resources, Supervision, Project administration. **Xinsheng Dong:** Writing - Original Draft, Validation, Formal analysis, Visualization, Software, Methodology, Investigation, Conceptualization. **Jiong Yu:** Resources, Investigation, Software. **Shuangquan Li:** Investigation. **Xinyu Xiong:** Formal analysis. **Hongtao Shen:** Visualization.



## Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Data availability

Data will be made available on request.

## References

- [1] Lu J, Wu D, Mao M, Wang W, Zhang G. Recommender system application developments: a survey. *Decis Support Syst* 2015;74:12–32.
- [2] Kumar C, Kumar M. Session-based recommendations with sequential context using attention-driven LSTM. *Comput Electr Eng* 2024;115:109138.
- [3] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. *Adv Neural Inf Process Syst* 2017;30.
- [4] Kang W-C, McAuley J. Self-attentive sequential recommendation. In: 2018 IEEE international conference on data mining. IEEE; 2018, p. 197–206.
- [5] Li J, Wang Y, McAuley J. Time interval aware self-attention for sequential recommendation. In: Proceedings of the 13th international conference on web search and data mining. 2020, p. 322–30.
- [6] Wu L, Li S, Hsieh C-J, Sharpnack J. SSE-PT: Sequential recommendation via personalized transformer. In: Proceedings of the 14th ACM conference on recommender systems. 2020, p. 328–37.
- [7] Sun F, Liu J, Wu J, Pei C, Lin X, Ou W, et al. BERT4Rec: Sequential recommendation with bidirectional encoder representations from transformer. In: Proceedings of the 28th ACM international conference on information and knowledge management. 2019, p. 1441–50.
- [8] Cho SM, Park E, Yoo S. MEANTIME: Mixture of attention mechanisms with multi-temporal embeddings for sequential recommendation. In: Proceedings of the 14th ACM conference on recommender systems. 2020, p. 515–20.
- [9] Mehta S, Ghazvininejad M, Iyer S, Zettlemoyer L, Hajishirzi H. Delight: Deep and light-weight transformer. 2020, arXiv preprint arXiv:2008.00623.
- [10] Said A, Jain BJ, Narr S, Plumbbaum T. Users and noise: The magic barrier of recommender systems. In: User modeling, adaptation, and personalization: 20th international conference, UMAP 2012, Montreal, Canada, July 16–20, 2012. proceedings 20. Springer; 2012, p. 237–48.
- [11] Zhang H, Li Y, Ding B, Gao J. Practical data poisoning attack against next-item recommendation. In: Proceedings of the web conference 2020. 2020, p. 2458–64.
- [12] Lever J, Krzywinski M, Altman N. Points of significance: model selection and overfitting. *Nature Methods* 2016;13(9):703–5.
- [13] Zhou K, Yu H, Zhao WX, Wen J-R. Filter-enhanced MLP is all you need for sequential recommendation. In: Proceedings of the ACM web conference 2022. 2022, p. 2388–99.
- [14] Ke G, He D, Liu T-Y. Rethinking positional encoding in language pre-training. 2020, arXiv preprint arXiv:2006.15595.
- [15] Soliman SS, Srinath MD. Continuous and discrete signals and systems. Englewood Cliffs; 1990.
- [16] Zheng Y, Pan L, Qian J, Guo H. Fast matching via ergodic markov chain for super-large graphs. *Pattern Recognit* 2020;106:107418.
- [17] Rendle S, Freudenthaler C, Schmidt-Thieme L. Factorizing personalized markov chains for next-basket recommendation. In: Proceedings of the 19th international conference on world wide web. 2010, p. 811–20.
- [18] Huang J, Zhao WX, Dou H, Wen J-R, Chang EY. Improving sequential recommendation with knowledge-enhanced memory networks. In: The 41st international ACM SIGIR conference on research & development in information retrieval. 2018, p. 505–14.
- [19] Cui Q, Wu S, Liu Q, Zhong W, Wang L. MV-RNN: A multi-view recurrent neural network for sequential recommendation. *IEEE Trans Knowl Data Eng* 2018;32(2):317–31.
- [20] Zhao Y, Li C, Peng J, Fang X, Huang F, Wang S, et al. Beyond the overlapping users: Cross-domain recommendation via adaptive anchor link learning. In: Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval. 2023, p. 1488–97.
- [21] Zhang Y, Yin G, Dong H, Zhang L. Attention-based frequency-aware multi-scale network for sequential recommendation. *Appl Soft Comput* 2022;127:109349.
- [22] Du X, Yuan H, Zhao P, Qu J, Zhuang F, Liu G, et al. Frequency enhanced hybrid attention network for sequential recommendation. In: Proceedings of the 46th international ACM SIGIR conference on research and development in information retrieval. 2023, p. 78–88.
- [23] Touvron H, Bojanowski P, Caron M, Cord M, El-Nouby A, Grave E, et al. Resmlp: Feedforward networks for image classification with data-efficient training. *IEEE Trans Pattern Anal Mach Intell* 2022;45(4):5314–21.
- [24] Tolstikhin IO, Houlsby N, Kolesnikov A, Beyer L, Zhai X, Unterthiner T, et al. Mlp-mixer: An all-mlp architecture for vision. *Adv Neural Inf Process Syst* 2021;34:24261–72.
- [25] Li M, Zhao X, Lyu C, Zhao M, Wu R, Guo R. MLP4Rec: A pure MLP architecture for sequential recommendations. 2022, arXiv preprint arXiv:2204.11510.
- [26] Song J, Wang Y, Tang S, Zhang Y, Chen Z, Zhang Z, et al. Local-global memory neural network for medication prediction. *IEEE Trans Neural Netw Learn Syst* 2020;32(4):1723–36.
- [27] Xu C, Zhao P, Liu Y, Xu J, S. Sheng VSS, Cui Z, et al. Recurrent convolutional neural network for sequential recommendation. In: The world wide web conference. 2019, p. 3398–404.
- [28] Yang C, Qiao S, Kortylewski A, Yuille A. Locally enhanced self-attention: combining self-attention and convolution as local and context terms. 2021, arXiv preprint arXiv:2107.05637.
- [29] Liu H, Jing L, Wen J, Wu Z, Sun X, Wang J, et al. Deep global and local generative model for recommendation. In: Proceedings of the web conference 2020. 2020, p. 551–61.
- [30] He Z, Zhao H, Lin Z, Wang Z, Kale A, McAuley J. Locker: Locally constrained self-attentive sequential recommendation. In: Proceedings of the 30th ACM international conference on information & knowledge management. 2021, p. 3088–92.
- [31] Jiang J, Zhang P, Luo Y, Li C, Kim JB, Zhang K, et al. AdaMCT: adaptive mixture of CNN-transformer for sequential recommendation. In: Proceedings of the 32nd ACM international conference on information and knowledge management. 2023, p. 976–86.
- [32] Liu Q, Zeng Y, Mokhosi R, Zhang H. STAMP: short-term attention/memory priority model for session-based recommendation. In: Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining. 2018, p. 1831–9.
- [33] Li J, Ren P, Chen Z, Ren Z, Lian T, Ma J. Neural attentive session-based recommendation. In: Proceedings of the 2017 ACM on conference on information and knowledge management. 2017, p. 1419–28.
- [34] Zhou C, Bai J, Song J, Liu X, Zhao Z, Chen X, et al. Atrank: An attention-based user behavior modeling framework for recommendation. In: Proceedings of the AAAI conference on artificial intelligence. 2018, p. 4564–71.
- [35] Li M, Hsu W, Xie X, Cong J, Gao W. SACNN: Self-attention convolutional neural network for low-dose CT denoising with self-supervised perceptual loss network. *IEEE Trans Med Imaging* 2020;39(7):2289–301.

- [36] Ding M, Zhou Y, Chi Y. Self-attention generative adversarial network interpolating and denoising seismic signals simultaneously. *Remote Sens* 2024;16(2):305.
- [37] Cai R, Wu J, San A, Wang C, Wang H. Category-aware collaborative sequential recommendation. In: *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 2021, p. 388–97.
- [38] Chen L, Yang N, Yu PS. Time lag aware sequential recommendation. In: *Proceedings of the 31st ACM international conference on information & knowledge management*. 2022, p. 212–21.
- [39] Anderson JG, Hough SE. A model for the shape of the Fourier amplitude spectrum of acceleration at high frequencies. *Bull Seismol Soc Am* 1984;74(5):1969–93.
- [40] Staelin DH. Fast folding algorithm for detection of periodic pulse trains. *Proc IEEE* 1969;57(4):724–5.
- [41] Howard AG, Zhu M, Chen B, Kalenichenko D, Wang W, Weyand T, et al. Mobilenets: Efficient convolutional neural networks for mobile vision applications. 2017, *arXiv preprint arXiv:1704.04861*.
- [42] McAuley J, Targett C, Shi Q, Van Den Hengel A. Image-based recommendations on styles and substitutes. In: *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval*. 2015, p. 43–52.
- [43] Harper FM, Konstan JA. The movielens datasets: History and context. *Acm Trans Interact Intell Syst (Tiis)* 2015;5(4):1–19.
- [44] Hidasi B, Karatzoglou A, Baltrunas L, Tikk D. Session-based recommendations with recurrent neural networks. In: *4th international conference on learning representations, ICLR 2016, conference track proceedings*. 2016.
- [45] Yuan F, Karatzoglou A, Arapakis I, Jose JM, He X. A simple convolutional generative network for next item recommendation. In: *Proceedings of the twelfth ACM international conference on web search and data mining*. 2019, p. 582–90.
- [46] Fan X, Liu Z, Lian J, Zhao WX, Xie X, Wen J-R. Lighter and better: low-rank decomposed self-attention networks for next-item recommendation. In: *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 2021, p. 1733–7.
- [47] Zhu Y, Huang B, Jiang S, Yang M, Yang Y, Zhong W. Progressive self-attention network with unsymmetrical positional encoding for sequential recommendation. In: *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval*. 2022, p. 2029–33.
- [48] Jiang Y, Xu Y, Yang Y, Yang F, Wang P, Li C, et al. TriMLP: A foundational MLP-like architecture for sequential recommendation. *ACM Trans Inf Syst* 2024;42(6):1–34.
- [49] Long C, Yuan H, Fang J, Xian X, Liu G, Sheng VS, et al. Learning global and multi-granularity local representation with MLP for sequential recommendation. *ACM Trans Knowl Discov Data* 2024;18(4):1–15.
- [50] Zhao WX, Mu S, Hou Y, Lin Z, Chen Y, Pan X, et al. Recbole: Towards a unified, comprehensive and efficient framework for recommendation algorithms. In: *Proceedings of the 30th acm international conference on information & knowledge management*. 2021, p. 4653–64.
- [51] Zhao WX, Hou Y, Pan X, Yang C, Zhang Z, Lin Z, et al. RecBole 2.0: towards a more up-to-date recommendation library. In: *Proceedings of the 31st ACM international conference on information & knowledge management*. 2022, p. 4722–6.
- [52] Belletti F, Chen M, Chi EH. Quantifying long range dependence in language and user behavior to improve RNNs. In: *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining*. 2019, p. 1317–27.